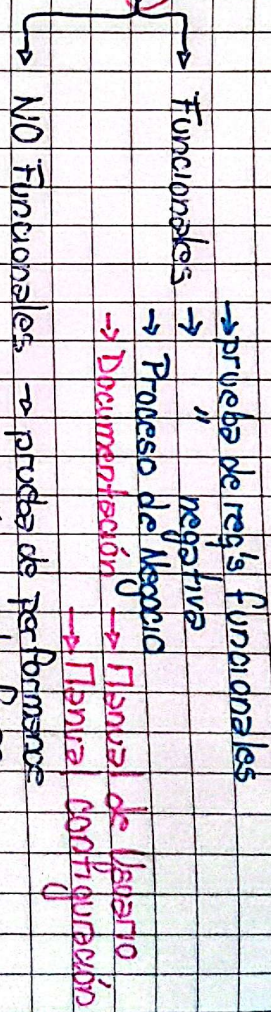


08/10 Testing / Prueba

Proceso [Pub]

Requerimientos



Analisis

Diseño

[35% del esfuerzo]

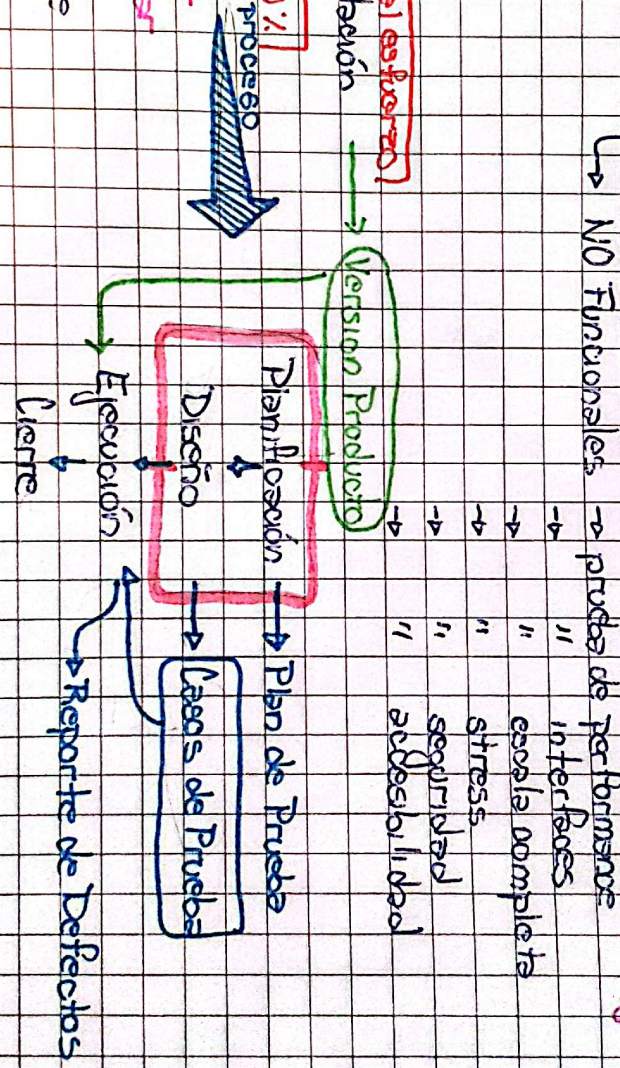
Implementación

[30% - 50%]

Prueba

La validar
La verificar

Despliegue



La Lido de Prueba

La Estrategia de Prueba

La Sin regresión

La con regresión

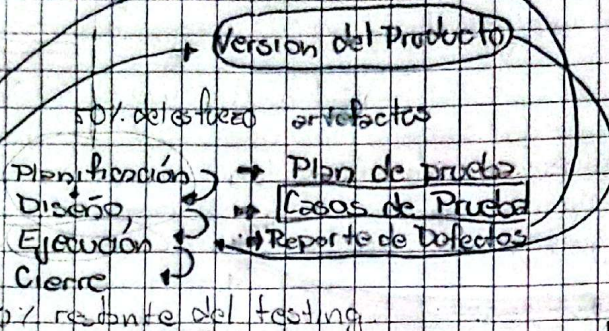
07/10

Testing/Pruebas

→ encontrar errores que se asumen en el código

Proceso [POD]

- Requerimientos
- Análisis
- Diseño
- Implementación
- **Prueba**
- Despliegue



Proceso cuyo propósito es detectar que se presume en el código defectos.

Puedo empezar a planificar y diseñar la prueba puedo llevarlo a Reg's por disminuir mi esfuerzo adelantando mi calendario

Los casos de prueba surgen de los requerimientos, por este motivo se adelantó los artefactos de Plan de prueba, casos de prueba y reportes de defectos siempre están en un tradicional y ágil (no importa la filosofía)

Conceptos

- Testing Metodológico vs AdHoc

testing metodológico → es el que usa un proceso y diseña casos de prueba

testing > AdHoc → no se acuerda

(NO tiene PROCESO)

ninguna estructura

- un error que no se puede reproducir, es un error.

- probar y probar

- no metodológico ni sistemático

Si no hay caso de prueba no tiene sentido probar.

Prueba

Validar proceso de vemos si hace lo que tiene que hacer

Verificar vemos que lo haga bien, pero hace lo que tiene que hacer.

para hacer los dos procesos necesita los casos de Prueba ya que para poder validar debe saber que debe hacer y esto se sustenta de los req's

también tenemos que tener en cuenta que el costo de validación es el error mismo de ahí es más caro y si es de verificación es menor

Error vs Defecto

La Falla

con consecuencias de los errores y defectos que tiene el su

Defecto: cuando se transfiere de una etapa anterior. No se generan en la misma etapa que es actualmente

Error: si no se trasfieren a otras etapas. Son menos los casos

Asamblea para poder mitigar: organización de partes

El costo más significativo es el Costo del Esfuerzo 30% de los costos

El testing se lleva entre el 30% - 50% del esfuerzo del total.

► implementación: debería llevarse el 35% del trabajo

No se hace tanto el testing exhaustivo se lleva mucho del costo del esfuerzo

Edad del defecto: mientras más edad tiene su costo tiene un decremento exponencial.

- La beta programming es la detección de los errores en ese momento es más barato como adelantar el code review, estamos controlando en el momento

si provocas un mal funcionamiento del sistema ocasionas una falla

no todos los errores ^{defecto} provocan fallas

- Severidad del Defecto → no tiene que ver lo que me cuesta corregirlo sino el impacto que genera al usar el sistema
- La Toleroquente - Invalidez
- La grave
- La leve
- La cosmética
- La ligera: es un consejo

esto viene del testing

- Prioridad Alta - media - Baja
- esto si viene del cliente

El testing siempre vinculado al proceso de desarrollo

Niveles de testing se prueban desde lo más chiquita a lo más grande

- Prueba de aceptación del usuario
 - Sistema o versión
 - Integración o Prueba de Interfaces
 - Prueba unitaria / Primer nivel de testing
- lo hacen el resto
→ lo hace quien lo implementa

En el workflow de despliegue

un desarrollador no debería probar su propio código
testing proceso de control

Definition of Done → pero errores conocidos

Ambientes de software

- desarrollo
 - pruebas
 - pre-producción
 - producción
- no tienen acceso a este ambiente.

pre-prod pruebas de aceptación de usuario

Asamblea

La gente de desarrollo no tiene permisos de acceso a logante ambiente de pruebas

Ciclo de pruebas es

Ciclo 0: el primero que se ejecuta en una versión

Ciclo 1

En terminos teoricos el ciclo 0 siempre esta y uno más, lo ideal es que sean 2 pero en la practica hay más ciclos.

haz que le de el OK por pasar por el proceso de aceptación de usuario

ciclo de pruebas por cada versión que entra al testing

• Estrategia a probar en cada ciclo de pruebas

↳ Sin regresión no se vuelve a probar los cambios

↳ Con regresión por cada error que se corrige se introducen 3 más.

Requerimientos Funcionales y No Funcionales // los dos son anticos

los pruebas No definidos con dificultades de probar

Tipos de Prueba

- foco del prod que hacen el prod

→ Funcionalidad

→ prueba de regla funcional (pass)

↳ y la prueba de reg's negativas. (Fail)

↳ pruebas proceso de negocio

↳ prueba de la documentación - Manual de Usuario, Manual de Configuración.

↳ NO Funcionales

→ Pruebas de performance

→ " de interfaces

→ escala completa. todo lo que me componen

much de sistema → prueba de estrés lo más bajo, pruebas con más alto que componen

y aceptación → pruebas de seguridad

→ " de accesibilidad

la verificación de cierre se da en el testing, no en el desarrollo.

que es un caso de prueba? situación exacta de lo que se quiere probar

Escenario (paso a paso)

+ todos de prueba

resultado esperado -> esta en las US con la Acceptation

+ necesito condiciones de setup o procedimientos

estrategia de minimización de defectos
defectos conocidos
deja de fabricar es un problema.

Atender la calidad de la prod

2) Protocolo del Testing es para asegurar la calidad de la prod. No
contar con todos los defectos. No
Testing hace control de calidad no asegura calidad